

Traffic Sign Classification with Transfer Learning: Studying the Effect of Augmentation and Synthetic Data

Team

Vishvesh Walunj, Mohneet Sandhu

1. Introduction

Image classification systems often perform well under clean conditions but degrade when images are noisy, blurred, partially occluded, or captured under poor lighting. In real-world recognition tasks such as traffic sign classification, this is a serious issue because signs may be viewed from different distances, angles, and environmental conditions. The goal of this project is to study how transfer learning can be adapted to a practical classification problem and how different data strategies affect both accuracy and robustness.

In this project, we fine-tuned an ImageNet-pretrained ResNet50 model on a 6-class subset of the German Traffic Sign Recognition Benchmark (GTSRB). We compared four training configurations: baseline training on original data only, training with augmentation, training with synthesized data, and training with synthesized data plus augmentation. In addition to standard accuracy evaluation, we also tested model robustness under distribution shifts such as noise, blur, occlusion, and low-light conditions.

The main question of this project is: **Which data strategy provides the strongest improvement in both classification accuracy and robustness for traffic sign recognition?**

2. Dataset

We used the **German Traffic Sign Recognition Benchmark (GTSRB)**, a real-world public dataset for traffic sign classification. The full dataset contains 43 traffic sign classes, but for this project we selected a smaller subset of 6 visually distinct classes so that experiments remained manageable and the results were easier to analyze.

The selected classes were:

- 00012 — Priority road

- 00013 — Yield
- 00014 — Stop
- 00017 — No entry
- 00025 — Road work
- 00038 — Keep right

These classes were chosen because they are diverse in shape, color, and symbol. For example, **Stop** is an octagon, **Yield** is a triangle, **No entry** is a red circular sign, and **Keep right** has a distinct directional symbol. This diversity made the subset suitable for comparing learning behavior and class confusion.

The official GTSRB dataset already provides separate training and test sets. We further divided the training portion into **training** and **validation** splits with no overlap. The split sizes used in the experiments were:

- Train: 7776 images
- Validation: 1944 images
- Test: 3210 images

This ensured a consistent setup across all four experiments.

3. Backbone Model and Transfer Learning Setup

We used **ResNet50 pretrained on ImageNet** as the backbone model. ResNet50 is a standard and well-established convolutional neural network for image classification. It was a suitable choice for this project because it provides strong transfer learning performance while still being practical to train and explain.

To adapt the pretrained model to our task, the final fully connected classification layer was replaced so that the network predicted the 6 selected traffic sign classes instead of the original ImageNet categories. The backbone feature extractor was frozen, and only the final classification layer was trained for the selected traffic sign task.

All images were resized to **224 × 224**, which matches the expected input size of ResNet50.

4. Experimental Configurations

We trained and evaluated the model under four required configurations.

4.1 Configuration 1: Baseline

The baseline model was trained using only the original training data from the selected 6-class subset. No augmentation and no synthetic data were used.

4.2 Configuration 2: Original Data + Augmentation

In this configuration, the original training data was used together with moderate training-time augmentation. The augmentation included:

- small rotation
- small translation
- brightness and contrast variation
- color jitter

Validation and test sets were kept unchanged so that the evaluation remained fair.

4.3 Configuration 3: Original Data + Synthesized Data

In this configuration, additional synthetic training images were generated from the original training split. Synthetic images were created using perturbations such as:

- Gaussian noise
- blur
- partial occlusion
- brightness changes

These new images were saved as additional training samples and combined with the real training data. Validation and test sets were unchanged.

4.4 Configuration 4: Original Data + Synthesized Data + Augmentation

In the final configuration, the model was trained on the combined real + synthetic training data while also applying augmentation during training. This was intended to test whether combining both strategies would produce the strongest result.

5. Evaluation Methodology

Each configuration was evaluated using:

- training loss curve
- validation loss curve
- training accuracy per epoch
- validation accuracy per epoch
- test accuracy
- classification report
- confusion matrix

Beyond standard performance, we also carried out a focused comparison between the **baseline model** and the **best-performing model**. This comparison included:

- side-by-side confusion matrix analysis
- error analysis of confusing classes
- robustness testing under corrupted inputs

The robustness evaluation tested the models under the following corruptions:

- clean
- noise
- blur
- occlusion
- low light

6. Results

6.1 Test Accuracy Across Configurations

CONFIGURATION	TEST ACCURACY
BASELINE	93.30%
AUGMENTED	94.14%
SYNTHESIZED	95.83%
SYNTHESIZED+AUGMENTED	95.64%

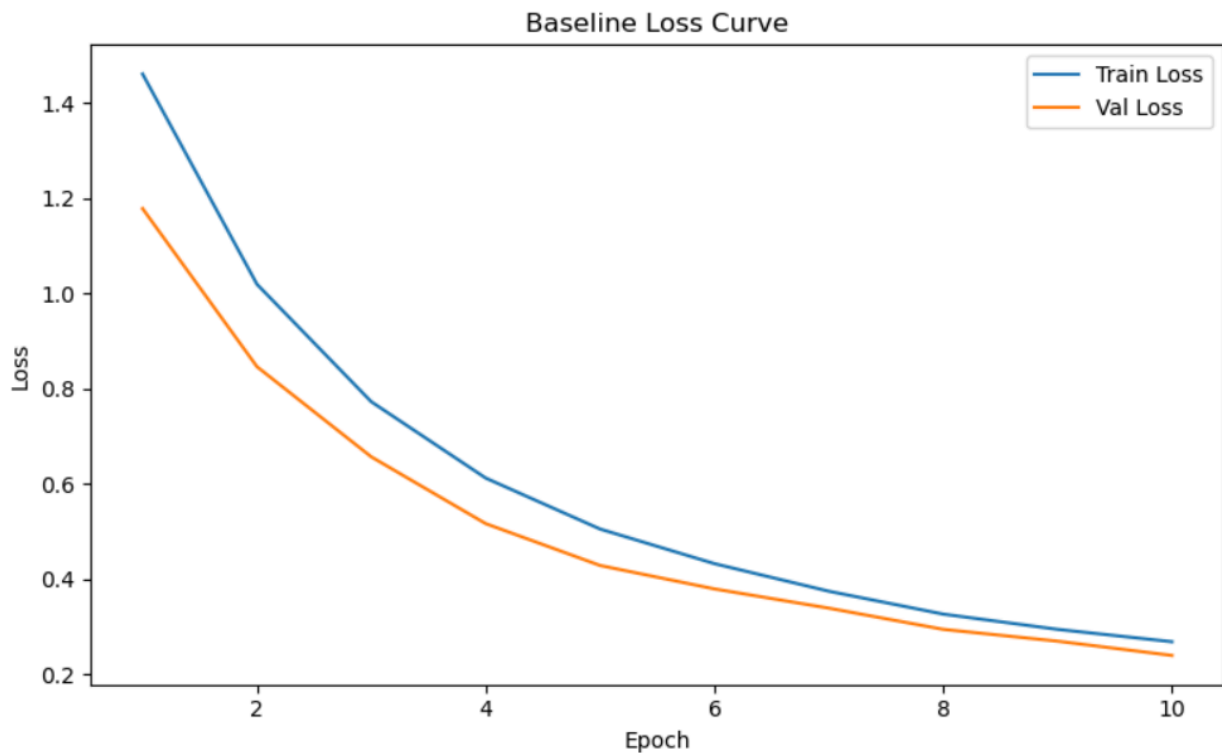
6.2 Best Model

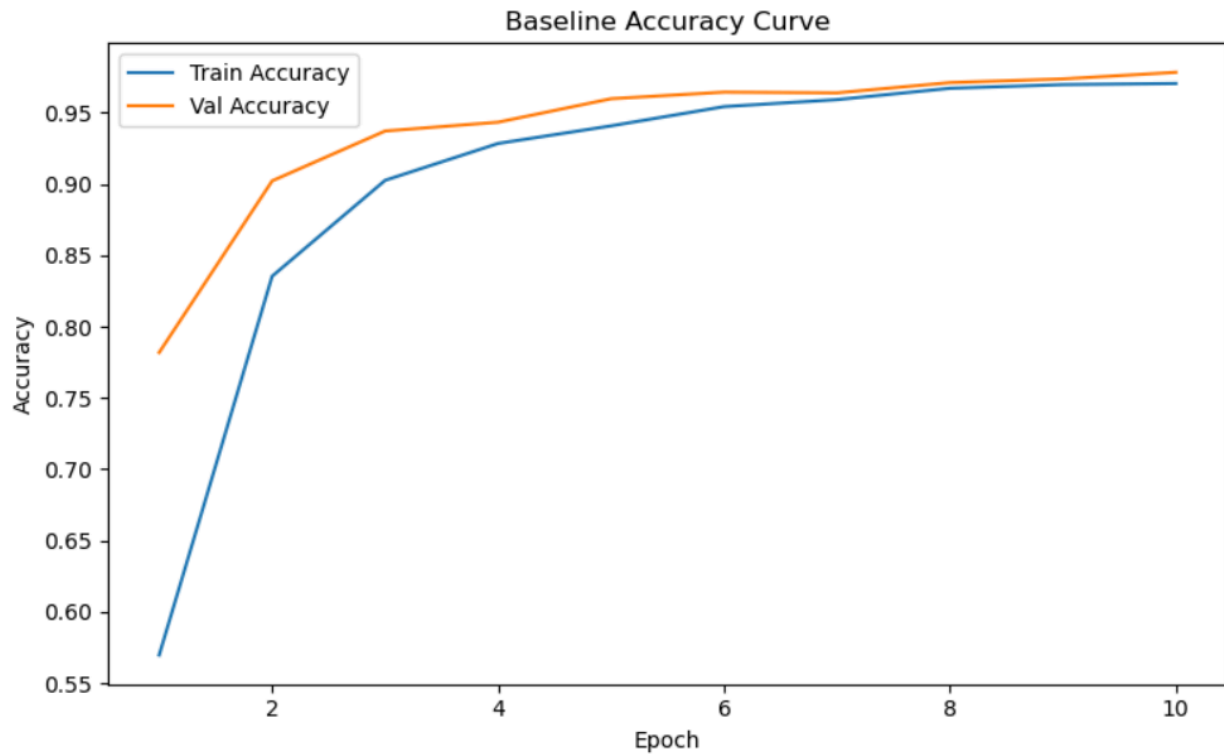
The best-performing model was **Configuration 3: Original Data + Synthesized Data**, with a test accuracy of **95.83%**.

6.3 Interpretation of Configuration Results

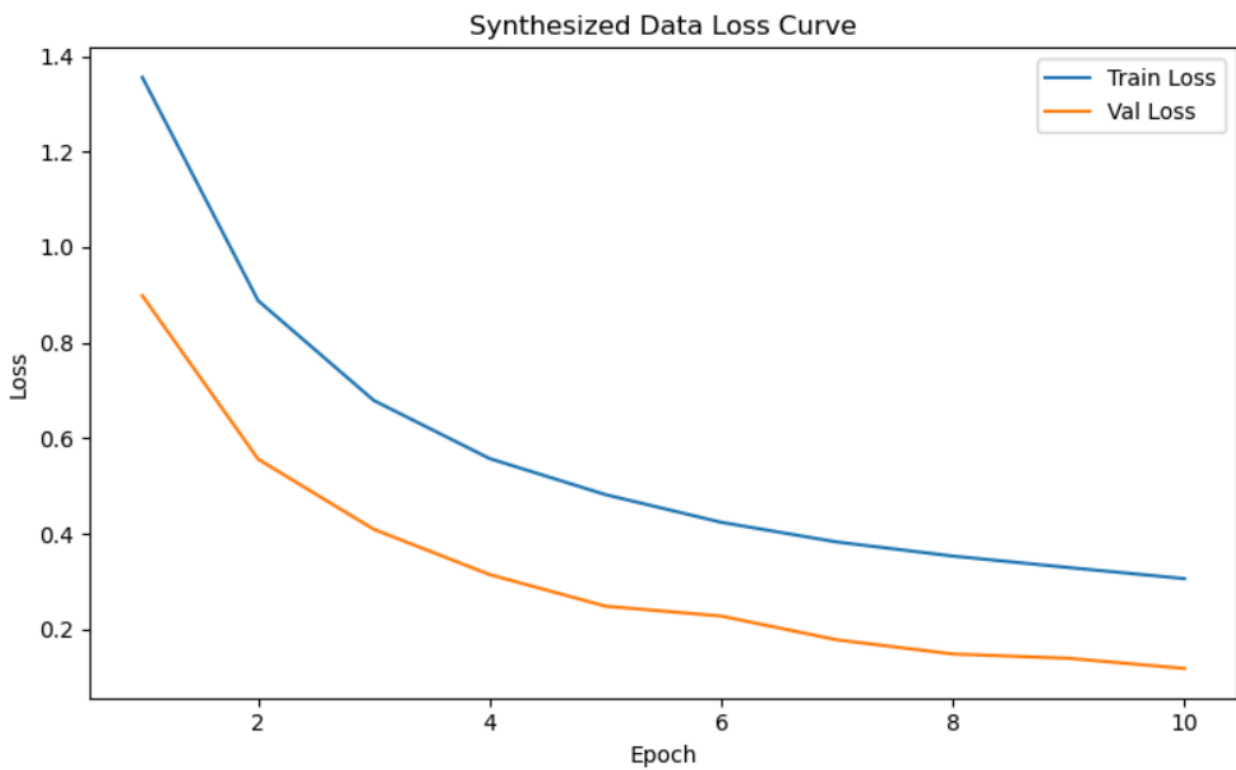
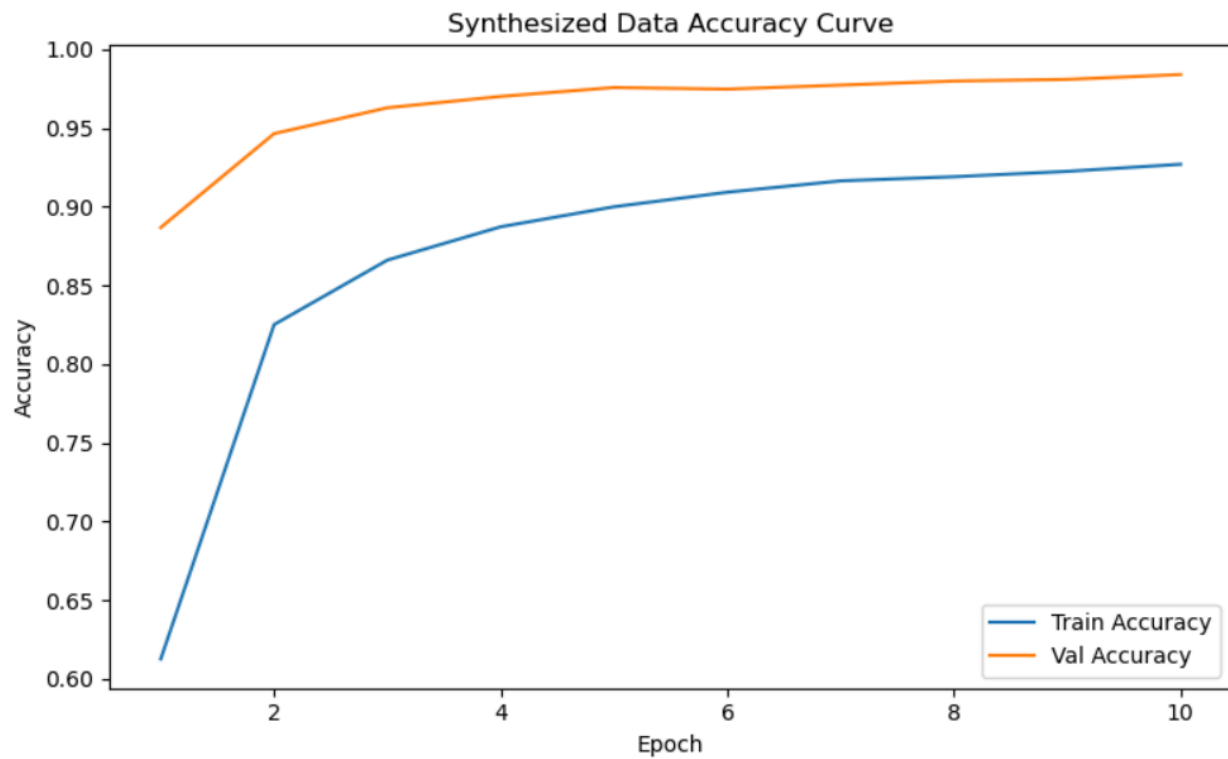
The baseline model already performed well, reaching **93.30%** test accuracy. This shows that transfer learning with ResNet50 was effective even when trained only on the original selected subset.

Adding training-time augmentation improved performance slightly to **94.14%**. This suggests that moderate augmentation helped the model generalize better by exposing it to variations in rotation, translation, and color during training.





The largest improvement came from adding synthesized data. The synthesized-data configuration achieved **95.83%**, outperforming both the baseline and the augmentation-only model. This indicates that saved synthetic examples provided useful additional variation that improved generalization more effectively than standard augmentation alone.



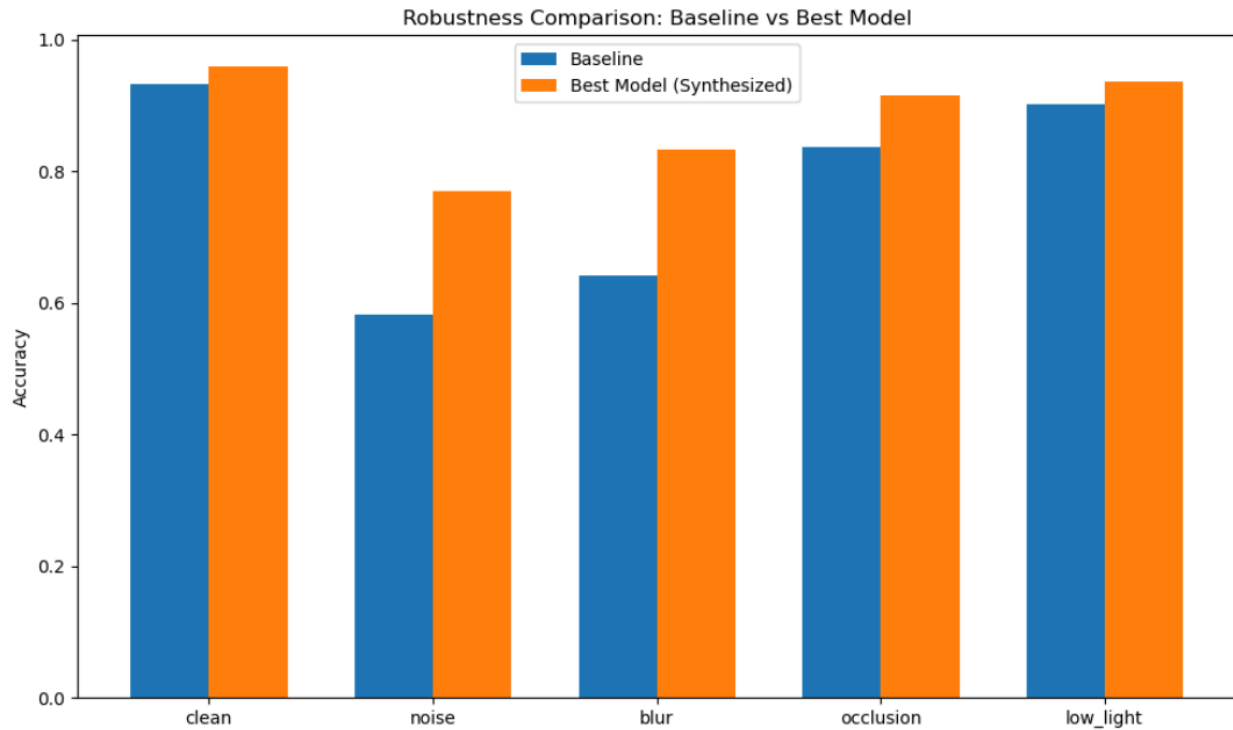
Interestingly, the combined synthesized + augmentation model achieved **95.64%**, which was still strong but slightly below the synthesized-only model. This suggests that while both techniques were beneficial, the combination may have introduced slightly more variation than needed for this particular subset. In other words, synthesis alone was already strong enough to produce the best overall result.

7. Robustness Testing

To better understand real-world reliability, we compared the **baseline model** and the **best model (synthesized-data model)** under several corrupted test conditions.

7.1 Robustness Results

CORRUPTION	BASELINE	BEST MODEL (SYNTHESIZED)
CLEAN	93.30%	95.83%
NOISE	58.13%	76.98%
BLUR	64.17%	83.27%
OCCLUSION	83.58%	91.43%
LOW LIGHT	90.25%	93.68%



7.2 Interpretation of Robustness Results

The synthesized-data model outperformed the baseline under every tested condition. The largest improvements appeared under **noise** and **blur**:

- Noise: +18.85 percentage points
- Blur: +19.10 percentage points

These results are important because they show that synthetic training examples significantly improved robustness under more difficult image degradations.

Occlusion also showed a clear improvement:

- Occlusion: +7.85 percentage points

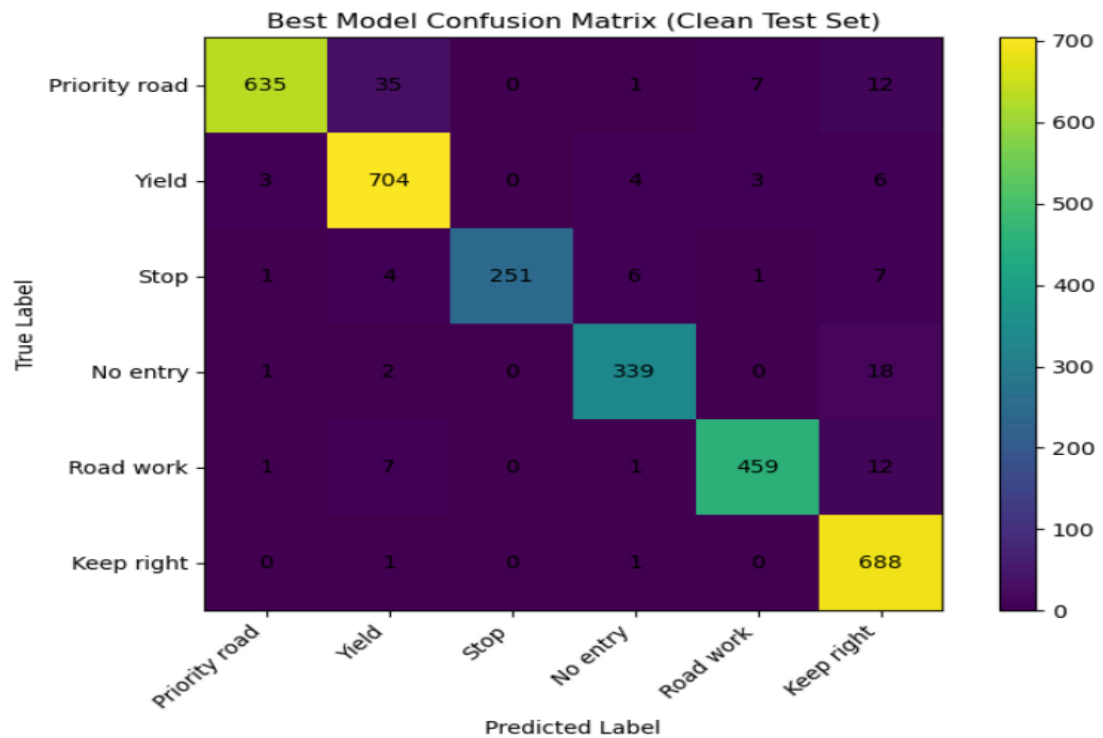
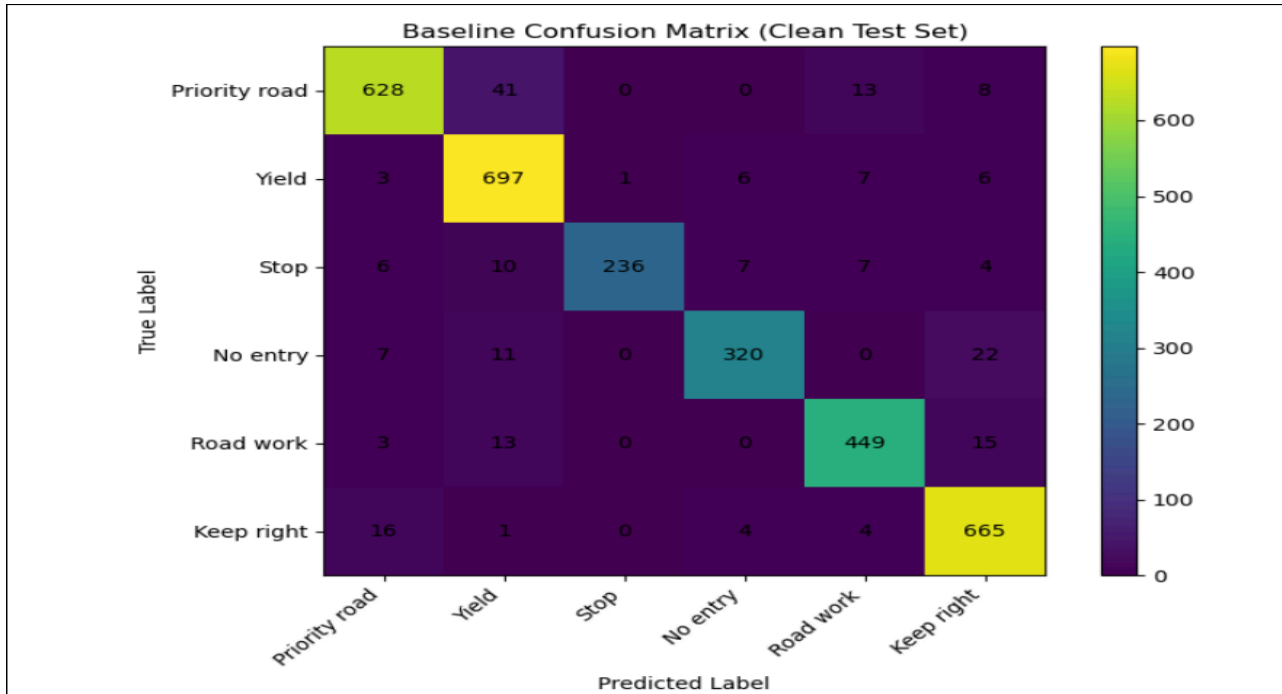
Low-light conditions caused a smaller but still meaningful gap:

- Low light: +3.43 percentage points

This suggests that the synthesized-data model learned more robust features than the baseline, especially when visual clarity was reduced.

8. Error Analysis

Error analysis was performed using the classification reports, confusion matrices, and robustness results.



One important observation is that **Stop** signs maintained very high precision, meaning that when the model predicted the Stop class, it was usually correct. However, recall for Stop was slightly lower than some other classes, showing that some Stop signs were still missed or confused under more difficult conditions.

Classes such as **Road work** and **Keep right** performed very strongly. This is likely because they have distinctive shapes, symbols, and color patterns, which makes them easier for the model to separate from other classes.

The most difficult corruptions for both models were **noise** and **blur**. This is reasonable because traffic sign recognition depends heavily on clear edges, boundaries, and symbols. Noise disrupts pixel-level detail, while blur weakens shape and symbol clarity. Since many signs depend on sharp visual structure, these distortions had the strongest negative impact.

Occlusion caused a smaller drop than blur or noise. This suggests that when part of a traffic sign remains visible, the model can still use partial cues such as color, shape, or symbol fragments to make a correct prediction.

Low-light conditions reduced performance less than noise and blur. This implies that brightness changes were easier for the model to tolerate than strong structural distortions.

Overall, the error analysis suggests that the main challenge in this task is not simply class diversity, but how well the learned features survive under image corruption.

9. Development Challenges

Several practical issues were encountered during the project.

First, Python environment setup caused repeated problems. Package installation conflicts and version mismatches made the initial setup difficult. These had to be resolved before training could proceed smoothly.

Second, dataset paths and folder structure required careful handling. Because the project relied on multiple scripts, consistent path definitions were necessary to avoid file-not-found errors and to ensure that outputs were saved correctly.

Third, maintaining fairness across all four configurations required discipline. The training and evaluation pipeline had to remain consistent, with only the intended component changed in each experiment. This is one reason why several `.py` files in the project look similar. The shared structure was intentional so that comparisons across configurations remained scientifically valid and easy to interpret.

Finally, saving outputs properly required debugging. It was important to make sure that accuracy curves, loss curves, confusion matrices, and CSV metric files were correctly generated for each configuration.

10. Acknowledgment and Development Notes

During this project, **Google Gemini** was used mainly for conceptual understanding of technical ideas. It helped clarify topics such as the differences between baseline training, augmentation, and synthesized data, proper train/validation/test splitting, the interpretation of confusion matrices, and the design of robustness testing under noise, blur, occlusion, and low-light conditions.

Some parts of the project were initially confusing, especially understanding how to keep all four experiment configurations consistent while changing only the required part of the pipeline. Additional difficulties included environment setup, file path management, saving evaluation outputs, and debugging certain functions related to synthetic image generation and evaluation scripts. These issues were gradually resolved through repeated testing, debugging, and result verification.

11. Conclusion

This project showed that transfer learning with ResNet50 is effective for traffic sign classification on a selected GTSRB subset. More importantly, it showed that the choice of data strategy significantly affects both standard accuracy and robustness.

Among the four configurations, the best result came from **training with synthesized data**, which achieved 95.83% test accuracy. This model also consistently outperformed the baseline under all corruption settings, with especially large gains under noise and blur.

The main conclusion is that **saved synthetic training examples were the most valuable improvement strategy for this task**. While augmentation also helped, synthesis provided the strongest gains in both clean accuracy and robustness.

In future work, the project could be extended by:

- using more GTSRB classes
- fine-tuning deeper layers instead of only the final classification layer
- trying lighter backbones such as MobileNet for faster inference

- evaluating more advanced synthetic generation methods
- exploring stronger robustness benchmarks

Overall, the project successfully demonstrated how transfer learning, augmentation, and synthesized data interact in a practical real-world image classification task.

12. Files Produced

The project repository includes:

- source code for all four configurations
- robustness testing scripts
- plots for loss and accuracy
- confusion matrices
- classification reports
- metrics CSV files
- a small test-data folder for reproducibility
- the original project proposal